

TDiFf: Detecting Bugs in DataFrame Systems via Transferred DBMS Test Cases

Jiaxin Hu
Xiamen University
School of Informatics
Xiamen, China
hujiaxin@stu.xmu.edu.cn

Shaowei Chen
Fujian Agriculture and Forestry University
College of Computer and Information Science
Fuzhou, China
chenshaowei@fafu.edu.cn

Rongxin Wu
Xiamen University
School of Informatics
Xiamen, China
wurongxin@xmu.edu.cn

Abstract

DataFrame systems are core components of modern data analytics platforms and achieve high performance through various optimization techniques. However, these optimizations can introduce subtle defects that are difficult to detect, undermining system reliability. Existing testing approaches primarily rely on manually written unit tests or automated unit test generation tools, which often produce invalid or low-quality test cases and have limited ability to exercise complex API sequences and optimization-induced behaviors.

In this paper, we present TDiFf, a novel LLM-based testing framework for DataFrame systems. TDiFf leverages the database-like characteristics of DataFrame systems by translating SQL queries into Python DataFrame test cases for differential testing. To address limitations of LLMs in mapping SQL constructs to DataFrame APIs and preserving semantic consistency, TDiFf incorporates feature knowledge extracted from SQL and DataFrame API documentation. In addition, it utilizes data flow graphs derived from SQL query plans to guide the generation of syntactically valid and semantically consistent DataFrame API sequences.

We evaluate TDiFf on six widely used DataFrame systems: *Pandas*, *Dask*, *CuDF*, *Modin*, *PySpark*, and *Polars*. The results show that more than 99% of the generated DataFrame test cases are valid. Using these test cases, TDiFf uncovers 35 unique and previously unknown bugs confirmed by developers, and outperforms state-of-the-art baseline approaches by 2.5× to 14×.

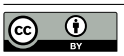
CCS Concepts

• **Software and its engineering → Software testing and debugging**.

Keywords

DataFrame Systems, Bug Detection

ACM Reference Format:
Jiaxin Hu, Shaowei Chen, and Rongxin Wu. 2026. TDiFf: Detecting Bugs in DataFrame Systems via Transferred DBMS Test Cases. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3832783.3837548



This work is licensed under a Creative Commons Attribution 4.0 International License.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3837548

1 Introduction

DataFrame systems serve as foundational infrastructure for modern data analytics, providing high-level abstractions for structured data manipulation across diverse domains including finance, bioinformatics, and scientific research [50, 51, 61, 68]. While *Pandas* [53] established the initial standard for Python data analysis [48], subsequent systems such as *Dask* [17], *CuDF* [20], *Modin* [19], *PySpark* [13], and *Polars* [23] have emerged to address scalability requirements through sophisticated optimization techniques including parallel execution, lazy evaluation, and hardware acceleration [21, 25, 58, 64]. However, the complexity of these optimizations frequently introduces subtle behavioral inconsistencies [32, 62, 87, 89], making comprehensive testing both critical and challenging.

Existing testing approaches primarily rely on manually written unit test suites or automated unit test generation tools. Although comprehensive unit test suites typically cover individual APIs, real-world bug reports reveal that many failures originate from interactions between multiple APIs. For instance, *issue #19772* [80] in *Polars* was triggered by the interaction between `DataFrame.join()` and `DataFrame.filter()` under lazy evaluation, despite both APIs passing their respective unit tests. This case indicates that unit tests alone are insufficient for detecting bugs arising from complex API sequences. Furthermore, the extensive API surfaces of DataFrame systems [14, 25, 54] make manual test construction impractical, while automated tools like *Pynguin* [45] often generate invalid or trivial test cases that fail to exercise meaningful API interactions. Consequently, more effective testing methodologies tailored to DataFrame system characteristics are needed.

A seemingly viable alternative is to test DataFrame systems directly through their SQL interfaces. However, SQL-interface testing cannot replace API-level testing because SQL support in existing DataFrame systems is often auxiliary and incomplete. First, existing SQL interfaces do not fully exercise DataFrame API behavior. Pandas-compatible systems such as *Dask*, *CuDF*, and *Modin* lack native SQL-interface support [15, 18, 21]. External SQL layers such as *pandasql* are outdated, with the latest release in 2016 [55]. For *Polars*, the DataFrame interface is primary, and new features are typically added to the expression API first [24]. *Spark* SQL mainly exercises the SQL engine rather than the native *PySpark* DataFrame API or the *Pandas* API on *PySpark* [14]. Second, in practice, fewer than 4% of the generated SQL queries can be successfully executed and used for testing by *PySpark* and *Polars*. Third, among the 42 bugs detected by TDiFf, 27, more than 64%, cannot be detected through SQL-interface testing. These findings motivate a testing strategy

```
1 # Original SQL Query Statement :
2 > SELECT (CASE 1 WHEN 1 THEN t1.c0 END) FROM t1 ORDER BY t1.c0 DESC;
3 # DataFrame Test Case Directly Transferred by LLMs :
4 > t1.select(pl.col('c0')).sort('c0', descending=True)
5 # DataFrame Test Case Transferred by LLMs with Data Flow Graphs :
6 > t1.select(pl.when(pl.lit(1) == pl.lit(1)).then(pl.col('c0')).otherwise(pl.lit(None)).alias('c0')).sort('c0', descending=True)
```

Figure 1: An example of a DataFrame test case transferred by LLMs directly and with data flow graphs, respectively.

that reuses mature SQL test generation while preserving API-level execution, leading to our SQL-to-DataFrame translation approach.

A fundamental insight underlying this work is the conceptual similarity between DataFrame systems and Database Management Systems (DBMSs) [61, 82]. DataFrame API sequences frequently correspond functionally to SQL queries, and both systems target analytical workloads characterized by extensive scanning and aggregation operations on columnar data [10, 68]. This similarity enables the translation of SQL queries into equivalent DataFrame API sequences, despite the differing interface paradigms (SQL versus DataFrame APIs). Meanwhile, the DBMS testing community has developed robust tools for generating high-quality SQL queries that effectively uncover bugs in DBMS implementations [62, 63, 65, 86]. These queries encode complex syntactic structures and rich semantics, making them suitable sources for generating meaningful DataFrame test cases. Recent advances in Large Language Models (LLMs) have demonstrated strong capabilities in code translation across programming languages [66, 88], offering a promising pathway to automate the translation from SQL queries to DataFrame API sequences. Utilizing LLMs for this translation avoids the need to handcraft a large number of transformation rules, thereby reducing manual engineering effort and enabling scalable application of mature DBMS testing techniques to DataFrame systems.

However, direct application of LLMs to this translation task faces two primary obstacles. First, LLMs often lack detailed knowledge of the precise correspondence between SQL constructs and DataFrame API functions, leading to syntactic errors. For example, an LLM might incorrectly translate the SQL function `argmin(c0,1)` into `F.argmin('c0',F.lit(1))` for *PySpark*, resulting in a syntax error, as the correct equivalent is `F.min_by('c0',F.lit(1))`. Second, LLMs exhibit limited understanding of the semantic alignment between SQL queries and DataFrame API sequences, which can result in partially lost, incorrectly ordered, or semantically inequivalent translations, thereby compromising bug detection effectiveness. For instance, Figure 1 shows that an LLM may directly translate the SQL conditional expression: `CASE 1 WHEN 1 THEN t1.c0 END` into the simplified DataFrame API call: `pl.col('c0')`, thereby eliminating semantically rich logic from the generated DataFrame API sequence. This loss of conditional logic undermines semantic fidelity. Conversely, the correct and complete translation, `pl.when(pl.lit(1) == pl.lit(1)).then(pl.col('c0')).otherwise(pl.lit(None))`, is exactly the expression that triggered a confirmed bug (*issue #25211* [76]) in *Polars*.

To address these limitations, we propose TDiFf, a testing framework that uses LLMs to translate SQL queries into DataFrame test cases for differential testing. Our design addresses these two challenges. First, to mitigate LLMs' lack of fine-grained correspondence knowledge between SQL constructs and DataFrame APIs, we build feature knowledge bases from official documentation using web crawlers and derive feature mapping tables with LLMs. During translation, SQL queries are parsed into features, and their correct DataFrame API mappings are retrieved from this feature mapping table, thereby preventing syntactic errors such as the `argmin` to `min_by` example. Second, to overcome LLMs' limited understanding of semantic alignment and execution order, we derive data flow graphs from SQL query plans. These graphs encode operator dependencies and logical execution sequences, providing guidance for generating DataFrame API sequences that are semantically coherent, complete, and correctly ordered, as illustrated by the example (*line 6*) in Figure 1. By integrating these two complementary forms of knowledge into the LLM prompt, TDiFf effectively bridges the gap between the declarative semantics of SQL queries and the procedural, order-sensitive nature of DataFrame API sequences, enabling the generation of valid, semantically equivalent test cases for differential testing. In addition, differential testing detects bugs by comparing execution results across systems with different optimization strategies (e.g., distributed and GPU execution) and their corresponding native, non-optimized counterparts using identical DataFrame test cases.

We implement TDiFf using GPT-5-mini and evaluate it on six widely used DataFrame systems, namely *Pandas*, *Dask*, *CuDF*, *Modin*, *PySpark*, and *Polars*. Experimental results demonstrate that over 99% of generated test cases are syntactically valid and executable. Moreover, TDiFf detects 42 unique bugs, of which 35 have been confirmed by developers as previously unknown, and 24 have already been fixed, outperforming state-of-the-art baseline approaches by a factor of 2.5× to 14× in bug detection.

In summary, we make the following contributions:

- We introduce a novel approach for generating effective test cases for DataFrame systems by translating SQL queries into equivalent DataFrame API sequences.
- We design an LLM-guided transfer approach that integrates feature knowledge and data flow graphs to improve both syntactic and semantic correctness during SQL-to-DataFrame translation.
- We implement our approach as TDiFf, a testing framework for DataFrame systems, and evaluate it on six widely used DataFrame systems. TDiFf achieves over 99% test case validity and uncovers 35 unique, developer-confirmed, previously unknown bugs. The source code and detected bug list of TDiFf are on Zenodo [35].

2 Background and Motivation

This section introduces the background of DataFrame systems and relevant testing techniques. It further analyzes limitations of existing testing approaches and motivates the key ideas of this work.

2.1 Introduction to DataFrame Systems

DataFrame Systems. DataFrame systems can provide a column-oriented abstraction for data analytics and are widely used in modern data processing pipelines [49, 57]. A DataFrame is a two-dimensional data structure consisting of rows and schema-defined columns, where each column is associated with a data type and schemas are often inferred at runtime [82]. Based on this abstraction, DataFrame systems expose high-level APIs that support relational operations such as selection, projection, joins, and aggregation [48].
DataFrame Systems Under Test. This work considers six representative DataFrame systems that are widely used in practice.

Pandas [53] is a Python library that popularized the DataFrame abstraction and provides flexible data structures for tabular data manipulation [48]. While Pandas is commonly used for small- and medium-scale data analysis, its in-memory execution model limits scalability for large datasets [49]. To address this limitation, several alternative systems have been developed.

Dask [17] extends Pandas DataFrames to support out-of-core and distributed execution. It partitions data across memory and disk and executes computations using a task scheduler [15, 64].

CuDF [20] provides a GPU-accelerated DataFrame implementation with a Pandas-like API. It enables tabular data processing on GPUs and is commonly used in GPU-based analytics pipelines [21].

Modin [19] is designed as a drop-in replacement for Pandas [18]. It parallelizes DataFrame operations across multiple CPU cores or distributed backends while preserving API compatibility [58, 59].

PySpark [13] is the Python interface of Apache Spark. Its DataFrame abstraction represents distributed data organized into named columns and supports relational-style operations [2]. *PySpark* exposes both native DataFrame APIs and Pandas APIs [14]. Spark RAPIDS [22] further enables GPU acceleration for workloads.

Polars [23] is a DataFrame library implemented in Rust. It provides native DataFrame APIs and supports parallel execution, lazy evaluation, and optional GPU acceleration [25].

Overall, these DataFrame systems fall into three categories: (1) Pandas-compatible systems (*Pandas*, *Dask*, *CuDF*, *Modin*, and *Pandas* on *PySpark*) [6], (2) *PySpark*, and (3) *Polars*. Differences in query semantics, data types, optimization strategies, and execution models (e.g., parallel, distributed, lazy, and heterogeneous execution) make systematic correctness testing necessary.

2.2 Introduction to OLAP DBMSs

OLAP DBMSs. Online Analytical Processing (OLAP) Database Management Systems (DBMSs), such as DuckDB [60], are designed to support analytical workloads involving scans, joins, and aggregations over large datasets [10]. This design differs from Online Transaction Processing (OLTP) systems, which focus on transactional workloads [7].

Relation to DataFrame Systems. OLAP DBMSs exhibit several similarities to DataFrame systems. Both target analytical workloads, adopt column-oriented data layouts, and apply query optimizations to improve performance [44, 57]. Although they expose different interfaces (SQL versus DataFrame APIs), this similarity enables the transfer of SQL queries into equivalent DataFrame API sequences.

2.3 DBMS Testing

DBMS Test Case Generator. In recent years, many generation-based testing techniques have been proposed for DBMSs, including SQLsmith [65], NoREC [62], TLP [63], and CODDTest [86]. These techniques generate SQL queries according to predefined rules or metamorphic relations and have revealed correctness issues in multiple DBMS implementations.

DataFrame systems expose relational-style APIs whose semantics largely align with SQL query features. This correspondence enables a SQL query to be systematically translated into an equivalent sequence of query-related DataFrame API calls. The resulting sequences are valid, executable inputs to DataFrame systems and can therefore serve as system-level test cases.

Based on this observation, we propose to leverage mature DBMS query-generation techniques to produce SQL queries and translate them into DataFrame API sequences for systematic testing of DataFrame systems.

Differential Testing. Differential testing identifies inconsistencies and correctness bugs by comparing the results of identical test cases across software systems or execution engines [47]. It is widely used to test DBMSs [28], deep learning libraries [12], and other complex software systems [9].

Meanwhile, DataFrame systems support multiple optimization strategies, as described in Section 2.1. This enables direct comparison of optimized and non-optimized execution results, revealing inconsistencies indicative of bugs. Therefore, we use differential testing as our test oracle.

2.4 Motivation

Despite extensive unit test suites, DataFrame systems continue to exhibit correctness issues. Public issue trackers of major systems [13, 17, 19, 20, 23, 53] contain a large number of reported bugs involving interactions among multiple APIs or execution optimizations.

For example, *Polars* reported a high-priority bug (*issue #19772* [80]) related to the interaction between `DataFrame.join()` and `DataFrame.filter()`. Although each API had dedicated unit tests [70, 71], the bug arose under lazy execution optimizations and reappeared after an initial fix. This example illustrates that unit tests focusing on individual APIs are insufficient to detect bugs triggered by API sequences and optimization behaviors.

Meanwhile, modern DataFrame systems expose hundreds of DataFrame-related APIs. Beyond manually written unit test suites, existing testing approaches mainly rely on automated unit test generation tools such as *Pynguin*. However, *Pynguin* often generates invalid or low-quality test cases and struggles to construct long, semantically meaningful API sequences. As a result, manually constructing complex and semantically valid API sequences remains labor-intensive, while automated generation must simultaneously ensure syntactic correctness and semantic consistency. These challenges highlight the need for systematic approaches to generating DataFrame API sequences for effective testing.

2.5 Key Idea

Because DataFrame systems exhibit database-like characteristics [61, 82], SQL queries and DataFrame API sequences are functionally similar. The DBMS testing community has developed mature and effective tools for generating high-quality SQL queries that successfully uncover bugs in DBMSs [62, 63, 65, 86]. This observation motivates us to leverage the success of DBMS testing by transferring SQL queries into equivalent DataFrame API sequences for testing. Meanwhile, Large Language Models (LLMs) have recently demonstrated remarkable capabilities in translation tasks across various programming languages [66, 88], suggesting a promising opportunity to employ LLMs to assist with automated test case transfer.

Figure 2 uses *Polars* as an illustrative example to highlight the limitations of existing approaches and to introduce our key idea. Unit test suites written by project developers adequately cover individual APIs like `DataFrame.sort()` and `DataFrame.limit()`, with all tests passing. Automated testing tools such as *Pynguin* can also generate unit tests for these APIs; however, the resulting test cases are

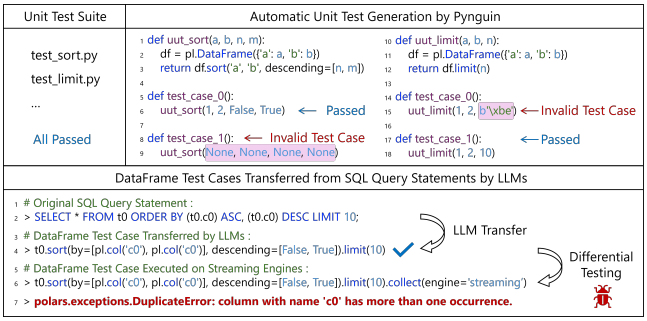


Figure 2: An illustrative example of our key idea.

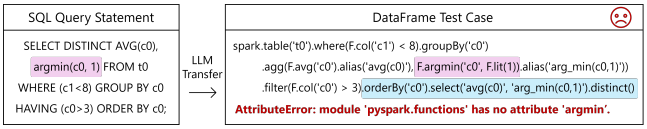


Figure 3: An example of an LLM failing to translate a SQL query into a DataFrame test case.

often trivial or contain invalid inputs (e.g., None values or opaque byte sequences). As a result, neither manual unit tests nor *Pynguin* can detect bugs that arise from specific API interactions. In contrast, our approach translates a SQL query into a valid DataFrame API sequence using an LLM and performs differential testing. This enabled the discovery of a bug (*Polars issue #25072* [74]) that occurs only when `DataFrame.sort()` and `DataFrame.limit()` are executed together under streaming mode (a memory optimization), but not under native execution.

In summary, using LLMs to translate SQL queries into DataFrame API sequences reduces manual effort and enables the reuse of mature DBMS testing techniques for DataFrame testing.

2.6 Challenges

Although SQL queries can be translated into DataFrame API sequences, directly applying LLMs to this transfer task introduces two key challenges.

First, LLMs may lack precise knowledge of feature correspondences between SQL constructs and DataFrame APIs, which can result in syntactically invalid API calls. As shown in Figure 3, the SQL function `argmin(c0,1)` may be translated into `F.argmin('c0', F.lit(1))`, which is invalid in *PySpark*, where the correct API is `F.min_by('c0',F.lit(1))`. Such errors prevent the generated API sequences from serving as executable test inputs.

Second, LLMs may fail to preserve semantic alignment between SQL queries and the generated DataFrame API sequences. In particular, partial loss or incorrect ordering of DataFrame operations can lead to computations that are not semantically equivalent to the original SQL query. This issue is especially problematic for complex queries that involve multiple operators and execution dependencies.

Taken together, these challenges limit the effectiveness of directly transferring SQL queries generated by DBMS testing tools into DataFrame test cases for bug detection.

To address the first challenge, TDiff incorporates feature knowledge extracted from both SQL and DataFrame API documentation to guide API selection during transfer. Specifically, it builds

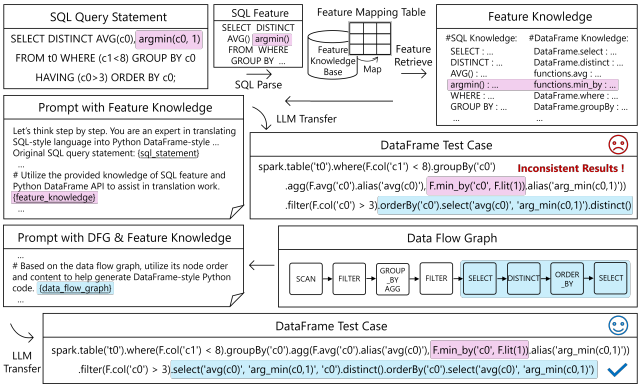


Figure 4: An illustrative example of challenges in translating a SQL query into a DataFrame test case using TDiff.

a feature mapping between SQL constructs and their corresponding DataFrame APIs and injects the relevant mappings into LLM prompts to reduce syntactic errors.

To address the second challenge, TDiff integrates data flow graphs from SQL query plans into the prompt. These graphs encode data dependencies and execution order, guiding the LLM to generate DataFrame API sequences that preserve the original SQL semantics.

Using the generated DataFrame API sequences as test cases, TDiff applies differential testing across different DataFrame systems and execution configurations, including parallel, distributed, lazy, and heterogeneous execution modes, to detect inconsistencies caused by system optimizations.

Figure 4 illustrates an example of how TDiff transfers SQL queries into DataFrame test cases. TDiff first retrieves feature mappings between DuckDB SQL and the *PySpark* DataFrame API, such as the correspondence between `argmin(c0,1)` and `F.min_by('c0',F.lit(1))`, to ensure syntactic validity. It then incorporates data flow information from the SQL query plan to prevent partial loss of DataFrame operations and guide the ordering of DataFrame API calls, resulting in a DataFrame test case that is both syntactically valid and semantically equivalent to the original SQL query.

3 Approach

This section presents the design of TDiff, an automated framework that leverages LLMs to transfer SQL queries generated for DBMS testing into DataFrame test cases for differential bug detection.

Figure 5 presents the overall workflow of TDiff, which consists of five core stages: (1) collecting diverse SQL queries from existing DBMS testing tools; (2) constructing feature knowledge to ensure syntactic correctness; (3) building data flow graphs to preserve semantic equivalence; (4) translating SQL queries into DataFrame API sequences using LLMs with structured prompts; and (5) performing differential testing on DataFrame systems using the translated test cases. This design addresses two key challenges, namely the lack of fine-grained mappings between SQL features and DataFrame APIs and the limited ability of LLMs to preserve semantic alignment.

3.1 SQL Query Collection

To obtain a rich and varied set of input SQL queries, we utilize existing DBMS testing tools as SQL generators. These tools are capable of producing SQL queries with complex syntactic structures and

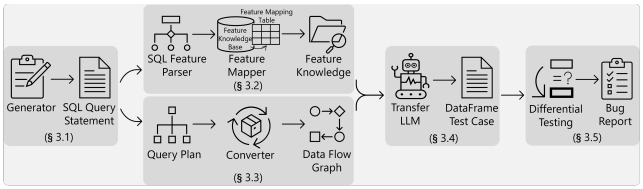


Figure 5: The overview of TDiFf.

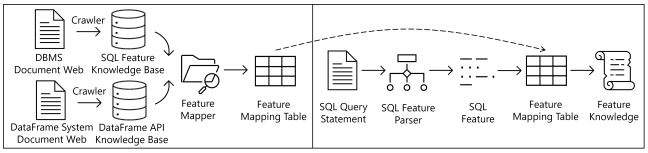


Figure 6: The design of offline feature knowledge construction and online feature knowledge retrieval.

rich semantics [62, 63, 65, 86], making their outputs well-suited for translation into expressive DataFrame test cases. For each generator, we collect multiple sets of SQL queries. The corresponding table schemas and data are also extracted and serialized into CSV format to be included as context in subsequent translation prompts.

3.2 Feature Knowledge Construction and Retrieval

To mitigate the risk of syntactic errors caused by LLMs' incomplete knowledge of the correspondences between SQL constructs and DataFrame APIs (Challenge #1 in Section 2.6), we generate and incorporate feature knowledge into the translation process.

Feature Knowledge Construction. We first construct feature knowledge bases offline by parsing the official documentation of the source DBMS (DuckDB [60]) and target DataFrame systems, as illustrated in Figure 6(a). The documentation is well structured and provides descriptions, rules, and examples for each feature, facilitating automated extraction. We develop web crawlers to collect this information. For SQL, features include *clauses* (e.g., `SELECT`, `WHERE`) and *functions* (e.g., `AVG()`, `argmin()`), whereas DataFrame features encompass all DataFrame API operations. For each feature, the extracted data includes its syntactic rule, description, and examples. We then remove invalid or duplicate entries. Because the target DataFrame systems fall into three categories (Pandas-compatible, *PySpark*, and *Polars*; Section 2.1), we build a separate API feature knowledge base for each category and one for DuckDB SQL.

Leveraging the conceptual similarity between SQL and DataFrame API sequences [61, 82], we construct feature mapping tables offline using LLMs to automatically map SQL features to functionally equivalent DataFrame API operations, as illustrated in Figure 6(a). We first embed the DataFrame API knowledge base into a vector database. For each SQL feature from the SQL feature knowledge base, we perform a vector similarity search to retrieve the top-k most similar DataFrame API candidates based on their rule and description embeddings. An LLM is then used to select the most syntactically appropriate match from these candidates. If no suitable match is found, the SQL feature is discarded. This process is repeated to construct a comprehensive feature mapping table for each of the three DataFrame API categories against DuckDB SQL.

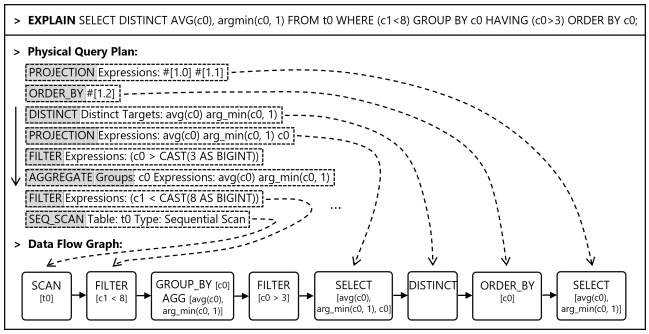


Figure 7: An example of converting a physical query plan to a data flow graph.

Using this procedure, TDiFf maps 268 of the 399 DuckDB SQL query features to Pandas-compatible APIs, 358 to *PySpark* APIs, and 301 to *Polars* APIs, corresponding to mapping ratios of 67.17%, 89.72%, and 75.44%, respectively. The remaining mismatches mainly reflect differences between SQL query features and DataFrame APIs. The unmapped SQL query features are mainly DuckDB-specific constructs, expressions, functions, and operators without direct DataFrame API counterparts. Conversely, the unmatched DataFrame APIs are mostly system-specific operations, such as index and axis manipulation, I/O and serialization, distributed execution, and Pandas/Arrow integration. Nevertheless, TDiFf focuses on translating SQL queries into query-related DataFrame test cases for bug detection and achieves relatively high feature-mapping coverage. Therefore, these remaining mismatches have limited impact on SQL-to-DataFrame translation and bug detection within our intended scope, especially because most unmatched DataFrame APIs are unrelated to query functionality.

To validate the correctness of the feature mapping table, we conducted a manual dual cross-verification process. Specifically, two independent postgraduate students separately checked the generated API list and SQL-to-DataFrame mappings against official documentation, API examples, and source-code implementations, and then cross-validated each other's results. Disagreements were resolved through discussion, while unresolved cases were further adjudicated by a third reviewer.

Feature Knowledge Retrieval. To retrieve fine-grained feature mapping tables online, complete SQL queries must be decomposed into their constituent features, as illustrated in Figure 6(b). We employ SQLGlot [69], a Python-based SQL parser, to parse each collected SQL query into a set of clause and function features. For instance, the SQL query: `SELECT DISTINCT AVG(c0), argmin(c0,1) FROM t0 WHERE (c1<8) GROUP BY c0 HAVING (c0>3) ORDER BY c0;` would be parsed into the feature set: {`SELECT`, `DISTINCT`, `FROM`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `AVG()`, `argmin()`}.

During online translation, the parsed SQL features are used to retrieve their DataFrame API counterparts from the offline-prepared feature mapping table. The corresponding documentation (rules, descriptions, examples) for both the SQL feature and the mapped DataFrame API is then formatted into plain text to be included in the LLM prompt. For example, the DuckDB function: `argmin()` would be mapped to the *PySpark* API: `F.min_by()`, and the relevant documentation for both would be supplied to the LLM.

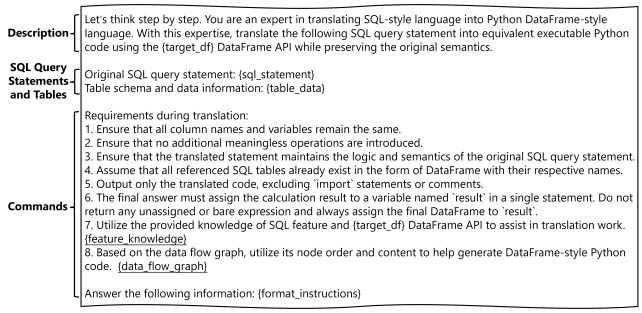


Figure 8: The details of the transfer prompt template. The template contains three main components, and parenthesized elements denote prompt variables.

3.3 Data Flow Graph Generation

To address the challenge of semantic misalignment, partial loss, and incorrect operation ordering in LLM-generated code (Challenge #2 in Section 2.6), we leverage data flow graphs (DFGs) derived from SQL query plans to guide the sequence of DataFrame API calls.

Query Plan Extraction. A physical query plan decomposes a SQL query into a tree of ordered relational operations and specifies their data dependencies [67]. By executing a SQL query prefixed with `EXPLAIN`, we obtain its serialized query plan from the DBMS.

Data Flow Graph. A Data Flow Graph (DFG) is a directed acyclic graph representing data dependencies among ordered operations [40]. The structured intermediate representation provided by this graph can bridge the gap between SQL queries and the operation sequences required by DataFrame APIs, thereby guiding LLMs to generate semantically consistent DataFrame test cases.

DFG Converter. Beyond the limitations of LLMs, it is observed that significant differences exist in the order between SQL queries and Python DataFrame API sequences. This discrepancy arises because SQL is a declarative, functionality-oriented language [4], whereas Python DataFrame systems rely on data-centric sequences of functional operations, leading to substantial differences in execution order even for semantically equivalent queries [82].

Based on the above observations, we leverage execution characteristics of DBMSs, such as query plans, to infer the ordering of operations and data dependencies.

However, many relational DBMSs adopt an operator-centric, pull-based execution model [30], such as DuckDB [60]. As a result, relational operations in the physical query plan are organized in a top-down order. This order is opposite to the bottom-up order of data-centric DataFrame API sequences. Consequently, the physical query plan needs to be converted into a bottom-up data flow graph to align with the execution semantics of DataFrame API sequences.

The following describes how a data flow graph is constructed from a physical query plan, as illustrated in Figure 7. First, the SQL query is prefixed with `EXPLAIN` to obtain its physical query plan. Next, operations, associated data (e.g., tables and rows), and data-related expressions are extracted in plan execution order. Operation names are then normalized, and redundant information is removed. The operations are connected in reverse order to form a directed acyclic graph. Finally, dot-to-ascii [29] converts the graph into plain text for inclusion in prompts.

Table 1: The overview of optimization features in DataFrame systems used for differential testing.

DataFrame Category	DataFrame System	Optimization Feature
Pandas-compatible	Pandas	Native
	Dask	Distributed Execution
	CuDF	GPU Execution
	Modin	Parallel Execution
	Pandas on PySpark	Distributed Execution
PySpark	PySpark	Native
	Spark-RAPIDS	GPU Execution
Polars	Polars	Native
		Lazy Execution
		Memory Optimization
		GPU Execution

3.4 LLM Transfer

With the feature knowledge and data flow graph prepared, we construct a structured prompt to guide the LLM in translating the SQL query into a DataFrame test case. The prompt, depicted in Figure 8, consists of three parts: (1) a *Task Description* outlining the translation objective and context; (2) the *Original SQL Query Statement* along with its table schema and sample data (in CSV format); and (3) a series of *Step-by-Step Commands* that integrate the retrieved feature knowledge and the data flow graph. These commands employ Chain-of-Thought (CoT) reasoning [81] to instruct the LLM to preserve semantics, column names, and variable bindings, and to avoid introducing redundant operations. This approach ensures the generated DataFrame API sequences are both syntactically correct and semantically faithful to the original SQL query.

3.5 Differential Testing

As described in Section 2.1, DataFrame systems adopt various optimization strategies, such as distributed execution and GPU execution. TDiff leverages these built-in optimization features to perform differential testing. Specifically, the generated DataFrame test cases are executed under different optimization configurations within the same system category. The results are then compared with a baseline result produced by the corresponding native execution, which applies no optimization or fewer optimizations. For example, TDiff can compare *Pandas* with *Dask*, or compare different *Polars* configurations with GPU execution enabled versus disabled. The specific optimization features used for differential testing in each DataFrame category are summarized in Table 1.

Before reporting discrepancies, TDiff applies a set of normalization and tolerance rules to reduce false positives, including floating-point tolerance, NULL/NaN normalization, type-casting checks, and row-order normalization. After these rules filter out acceptable differences, TDiff flags discrepancies, such as crashes, runtime errors, or logical inconsistencies, as potential bugs for manual inspection.

4 Evaluation

This section presents the experimental evaluation of TDiff, designed to address the following research questions:

RQ1: How effective and efficient is TDiff in transferring SQL queries into DataFrame test cases?

Table 2: The characteristics of DataFrame systems under test.

DataFrame System	Version	GitHub Stars	LOC	First Release
Pandas	2.3.3	47.7k	904k	2009
Dask	2025.11.0	13.7k	275k	2015
CuDF	25.10.00	9.5k	144k	2017
Modin	0.37.1	10.4k	256k	2018
PySpark	4.0.1	42.7k	89k	2012
Polars	1.35.2	37.1k	80k	2021

Table 3: Validity of DataFrame test cases translated by TDiFf across different SQL query generators. *Valid Proportion = (valid DataFrame test cases / total DataFrame test cases generated within six hours)* × 100%.

DataFrame Category	Pandas-compatible	PySpark	Polars	Total
SQLsmith	100%	99.51%	99.49%	99.66%
NoREC	100%	99.51%	99.49%	99.66%
TLP	100%	98.54%	98.99%	99.16%
CODDTest	100%	100%	100%	100%
Total Valid Proportion	100% (753/753)	99.39% (813/818)	99.50% (789/793)	99.62% (2355/2364)

RQ2: Can TDiFf detect previously unknown bugs in widely-used DataFrame systems?

RQ3: How does TDiFf compare to state-of-the-art baseline approaches in terms of bug detection capability?

RQ4: To what extent do feature knowledge and data flow graphs contribute to the effectiveness of transferring SQL queries into DataFrame test cases?

DataFrame Systems Under Test. We evaluate TDiFf on six popular open-source DataFrame systems: *Pandas*, *Dask*, *CuDF*, *Modin*, *PySpark*, and *Polars* (see Section 2.1 for detailed introduction). These systems represent the three major DataFrame categories identified in Section 2.1: (1) the Pandas-compatible category (*Pandas*, *Dask*, *CuDF*, *Modin*, and *Pandas* on *PySpark*); (2) the *PySpark* category; and (3) the *Polars* category. Detailed version and configuration information for each system is provided in Table 2.

Baselines. We compare TDiFf against five state-of-the-art testing approaches applicable to DataFrame systems:

(1) **FuzzyData** [61]: It is a native framework for generating and benchmarking DataFrame workflows. For comparison, we use its capability to randomly generate DataFrame API sequences, which are then used as test cases for differential testing.

(2) **LLM-powered Differential Unit Test (DUT*)**: We adapt the Differential Unit Test (DUT) concept [26, 84] for DataFrame systems. This baseline uses an LLM to directly generate unit test cases from DataFrame API documentation, which are then executed for differential testing.

(3) **LLM-assisted Pynguin (Pynguin*)**: *Pynguin* [45] is a state-of-the-art automated unit test generator for Python. This baseline uses an LLM to infer input types [41] and adapts DataFrame APIs into a form compatible with *Pynguin*, which then generates unit tests using the DynaMOSA algorithm [56] to detect bugs.

(4) **DAGger** [36]: It is a testing framework for data-intensive computing systems, including Pandas-compatible systems (*Dask*), *PySpark*, and *Polars*, and serves as a representative baseline for DataFrame system testing. It directly generates DataFrame API sequences by constructing directed acyclic graphs (DAGs) of operators, thereby enabling native and systematic testing.

(5) **DAGger-Diff**: It extends *DAGger* by applying its generated DataFrame API sequences to differential testing, aiming to expose inconsistencies across different execution backends.

Table 4: Average transfer time, execution time (under differential testing), token usage and cost using TDiFf.

DataFrame Category	Pandas-compatible	PySpark	Polars
Transfer Time	27.74s	25.91s	27.18s
Execution Time	0.93s	0.47s	0.04s
Input Tokens	6,972	8,225	4,951
Output Tokens	2,112	3,119	2,697
Total Tokens	9,084	11,344	7,648
Cost	$0.0064	$0.0090	$0.0072

SQL Query Collection. To evaluate TDiFf's sensitivity to the choice of SQL query generator and provide diverse inputs, we collect SQL queries from four generation-based DBMS testing tools: SQLsmith [16, 65], NoREC [62], TLP [63], and CODDTest [86]. Each tool generates an equal number of DuckDB SQL queries, which together form our evaluation dataset.

Environment. All experiments are conducted on a server with an 8-core Intel Core i7-11700 CPU@2.50GHz, 64GB of memory, and an NVIDIA GeForce RTX 4090 GPU (driver 580.95.05, CUDA 13.0). The operating system is Ubuntu 24.04 LTS with Linux kernel 6.8.0-31-generic. We use the OpenAI GPT-5-mini API (version `gpt-5-mini-2025-08-07`) [52] with the prompt structure described in Section 3.4. To ensure deterministic and reproducible outputs, the LLM temperature is set to 0.0. The reasoning effort parameter is set to a medium level to balance reasoning quality and time consumption during the translation process.

4.1 RQ1: Effectiveness and Efficiency

RQ1 evaluates the effectiveness and efficiency of TDiFf in translating SQL queries into DataFrame test cases. The quality and throughput of this translation directly impact the downstream bug detection capability of the framework.

Experimental Setup. Since there are three categories of DataFrame systems as described in Section 2.1, we configure TDiFf to translate DuckDB SQL queries into the corresponding three categories of DataFrame API sequences. For each category, we run TDiFf with a time budget of six hours to process SQL queries from the SQL dataset. A translated DataFrame test case is considered valid if it executes without errors on native settings (i.e., no optimization; see Table 1). This validity criterion ensures the generated test cases are suitable for subsequent differential testing.

Experimental Results. Table 3 summarizes the validity of test cases generated by TDiFf across the three DataFrame categories and four SQL query generators, thereby evaluating TDiFf's sensitivity to the choice of SQL query generator. The framework achieves a validity rate exceeding 99% overall, with a perfect 100% rate for the Pandas-compatible category. Across the four SQL query generators, the overall validity rates range from 99.16% to 100%. These consistently high rates indicate that TDiFf reliably produces valid test cases suitable for differential testing and is robust to the choice of SQL query generator.

Table 5: Classification of bug categories detected by TDiFf.

DataFrame Category	Crash Bug	Error Bug	Logical Bug
Pandas-compatible	1	1	13
PySpark	0	4	4
Polars	1	13	5
Total	2	18	22

Table 6: Classification of bug-triggering patterns identified by TDiFf via differential testing.

DataFrame Category	Trigger Pattern	Detected Bug
Pandas-compatible	Native vs. Distributed Execution	6
	Native vs. GPU Execution	6
	Native vs. Parallel Execution	3
PySpark	Native vs. GPU Execution	8
Polars	Native vs. Lazy Execution	1
	Native vs. Memory Optimization	4
	Native vs. GPU Execution	14

Table 4 reports the average translation time per SQL query and execution time per DataFrame test case under differential testing. TDiFf completes each translation in under 28 seconds on average, with specific averages of 27.74 seconds for Pandas-compatible APIs, 25.91 seconds for *PySpark*, and 27.18 seconds for *Polars*. This table also summarizes the average LLM token consumption and monetary cost per test case. During transfer, TDiFf consumes an average of 9,084 total tokens for Pandas-compatible APIs, 11,344 total tokens for *PySpark*, and 7,648 total tokens for *Polars*. These correspond to average costs of approximately $0.0064, $0.0090, and $0.0072 per test case, respectively. Overall, these results indicate that TDiFf can generate test cases with manageable computational overhead and LLM cost, supporting practical and scalable application.

Although TDiFf currently incurs moderate translation overhead because it invokes the LLM API sequentially, individual translations are independent and can be readily parallelized. Parallel API calls can substantially improve throughput. For example, 10-way parallelism can theoretically achieve up to a tenfold increase. Moreover, translated test cases can be cached and reused across testing runs, further amortizing the translation cost. Thus, the translation overhead does not hinder the practical scalability of TDiFf.

Answer to RQ1: TDiFf demonstrates high accuracy and acceptable time overhead in transferring SQL queries into DataFrame test cases, making it a practical approach for bug detection in DataFrame systems.

4.2 RQ2: Bug Detection

RQ2 investigates whether TDiFf can uncover previously unknown bugs in widely-used DataFrame systems, evaluating its practical effectiveness for real-world testing.

Experimental Setup. The experimental configuration follows the setup described in RQ1. For each DataFrame category, we executed TDiFf with a six-hour time budget for bug detection. Within this period, TDiFf first translates SQL queries into DataFrame test cases and then employs them for differential testing. Specifically, TDiFf compares execution results between systems with different optimization features and their corresponding native, non-optimized counterparts using the same test cases. After bug detection, we manually inspect all potential issues, remove duplicates, and report confirmed bugs to the corresponding development teams.

Table 7: Number of bugs detected by DataFrame test cases translated from different SQL query generators.

DataFrame Category	Pandas-compatible	PySpark	Polars	Total
SQLsmith	5	4	7	16
NoREC	4	1	5	10
TLP	3	2	3	8
CODDTest	3	1	4	8

Table 8: Summary of bugs detected by TDiFf.

DataFrame Category	Detected	Confirmed	Fixed	Not a Bug	Waiting
Pandas-compatible	15	9	3	1	5
PySpark	8	8	4	0	0
Polars	19	18	17	1	0
Total	42	35	24	2	5
Proportion	-	83% (35/42)	57% (24/42)	5% (2/42)	12% (5/42)

Experimental Results. As reported in Table 8, TDiFf detected a total of 42 bugs across six DataFrame systems, spanning all three DataFrame categories. These include 15 bugs in Pandas-compatible systems, 8 bugs in *PySpark*, and 19 bugs in *Polars*. Among these, 35 bugs (83% of those detected) have been confirmed by developers as unique and previously unknown, with 24 bugs (57%) already fixed. Meanwhile, 2 bugs (5%) have been identified by developers as duplicates or marked as `Won't Fix` (see Section 5.1 for details), and 5 bugs (12%) are pending developer confirmation.

Table 5 summarizes the types of bugs detected by TDiFf. The results include 2 crash bugs, 18 error bugs, and 22 logical bugs, indicating that most detected issues are semantic or behavioral inconsistencies and errors rather than system failures. This demonstrates TDiFf's ability to uncover subtle, non-trivial defects in DataFrame systems. Among the detected bugs, 20 bugs are revealed by direct execution of transferred DataFrame test cases, including 2 crashes and 18 runtime errors, whereas 22 logical bugs are exposed by differential testing between native and optimized execution. This suggests that SQL-to-DataFrame translation can independently generate effective bug-triggering test cases, while differential testing provides the necessary test oracle for identifying logical bugs.

Table 6 further illustrates that the majority of bugs are triggered by discrepancies between native execution and optimized execution modes. In Pandas-compatible systems, bugs arise from comparisons involving distributed, GPU, or parallel execution. For *PySpark*, inconsistencies are exclusively between native and GPU execution. In *Polars*, bugs are found when comparing native execution with lazy evaluation, memory optimization, or GPU execution. These findings indicate that optimization strategies, particularly GPU acceleration, parallel execution, lazy evaluation, and memory optimization, constitute a significant source of bugs in DataFrame systems. They also confirm that TDiFf effectively reveals such optimization-induced inconsistencies through differential testing.

Furthermore, we examine the sensitivity of TDiFf's bug-detection effectiveness to the choice of SQL query generator. Table 7 shows

Table 9: Comparison of bug detection and DataFrame API coverage (in parentheses) against baselines within six hours. *Note:* Since *DUT and *Pynguin** generate unit tests for individual APIs, they are set to target the same APIs covered by TDiFf and are thus excluded from the coverage comparison.**

DataFrame Category	Pandas-compatible	PySpark	Polars	Total
FuzzyData	3 (20)	-	-	3 (20)
DUT*	8	2	7	17
Pynguin*	6	1	5	12
DAGger	1 (25)	2 (82)	2 (23)	5 (130)
DAGger-Diff	2 (25)	1 (82)	5 (23)	8 (130)
TDiFf	**15 (95)**	**8 (90)**	**19 (70)**	**42 (255)**

that TDiFf detects bugs using SQL queries from all four generators. Although the yield varies from 8 to 16 bugs, TDiFf remains effective across the evaluated SQL query generators, suggesting that its bug-detection effectiveness is not tied to a particular generator.

Answer to RQ2: TDiFf detected a total of 42 bugs across multiple DataFrame systems, of which 35 were confirmed and 24 were fixed by developers, demonstrating its effectiveness in uncovering previously unknown bugs in real-world DataFrame systems.

4.3 RQ3: Comparison with Baselines

RQ3 evaluates the bug detection performance of TDiFf by comparing it with state-of-the-art baseline approaches.

Experimental Setup. We compare TDiFf against five baseline approaches: *FuzzyData*, *DUT**, *Pynguin**, *DAGger*, and *DAGger-Diff*. Each approach is allocated a six-hour time budget for bug detection per DataFrame category. Since *FuzzyData* only generates Pandas-compatible API sequences, we apply it only to systems in this category. *DUT** and *Pynguin** are general-purpose Python testing techniques, and we evaluate them across all three DataFrame categories. To ensure a fair comparison, we first identify the DataFrame APIs covered by the API sequences generated by TDiFf within its six-hour run. We then configure *DUT** and *Pynguin** to generate unit test cases targeting the same set of APIs, as both baselines primarily produce tests for individual API functions.

Experimental Results. Table 9 reports the bug-detection results and DataFrame API coverage of TDiFf and the baselines under a six-hour time budget. TDiFf substantially outperforms all baselines by detecting 42 bugs, corresponding to an improvement factor ranging from 2.5× to 14×. By comparison, the native DataFrame testing approaches, namely *FuzzyData*, *DAGger*, and *DAGger-Diff*, detect only 3, 5, and 8 bugs, respectively, whereas the unit-testing approaches, *DUT** and *Pynguin**, detect 17 and 12 bugs, respectively. In terms of DataFrame API coverage, *FuzzyData* covers 20 APIs, whereas *DAGger* and *DAGger-Diff* each cover 130 APIs, while TDiFf covers 95 Pandas-compatible APIs, 90 *PySpark* APIs, and 70 *Polars* APIs, for a total of 255 APIs, demonstrating substantially broader API coverage than the baselines.

We further analyze the factors underlying these results. *FuzzyData* generates numerous random API sequences but suffers from substantial duplication, limiting test diversity. *DUT** rapidly produces LLM-based unit tests, but its inputs often contain excessive empty, `NaN`, or `None` values and rely heavily on API documentation. It also rarely explores multi-API interactions, limiting its ability to expose complex bugs. *Pynguin** enhances test generation with search-based techniques and LLMs, but inaccurate type inference, weak semantic oracles, and limited support for complex API sequences reduce its effectiveness. Its focus on crash and error bugs further limits the detection of backend-specific inconsistencies. *DAGger* and *DAGger-Diff* must generate valid multi-API sequences from scratch and manually implement the complex API dependencies and schema constraints required by each sequence, making the process labor-intensive and limiting test diversity.

In contrast, TDiFf translates syntactically complex and semantically rich SQL queries into equivalent DataFrame API sequences. This approach mitigates semantic ambiguity, coverage limitations, and execution-model blindness, enabling detection of subtle cross-API and optimization-induced bugs across DataFrame systems.

Furthermore, we evaluate SQL-interface testing using four DBMS testing approaches, namely SQLsmith, NoREC, TLP, and CODDTest, under a shared six-hour budget equally divided among them. Since the evaluated versions of the Pandas-compatible systems do not provide usable SQL interfaces, SQL-interface testing cannot be applied to them. For *PySpark* and *Polars*, SQL support is also incomplete: only 3.92% and 1.79% of the generated SQL queries can be successfully executed and used for testing, respectively. Among the 42 bugs detected by TDiFf, 27 (64%), including 15 in the Pandas-compatible systems, 2 in *PySpark*, and 10 in *Polars*, cannot be detected through SQL-interface testing. These results highlight the necessity of SQL-to-DataFrame translation, which preserves the complex semantics of SQL queries while exercising DataFrame-specific execution paths that are inaccessible through SQL interfaces.

We further conduct differential testing between the source DBMS and the target DataFrame systems under the same six-hour budget by comparing the execution results of each SQL query with those of its translated DataFrame API sequence. After manual inspection, none of the observed discrepancies led to a confirmed bug. In contrast, TDiFf performs differential testing between native and optimized execution within the same DataFrame system, providing a more effective test oracle.

Answer to RQ3: Compared with the state-of-the-art baseline approaches, TDiFf can detect 2.5× to 14× as many bugs in DataFrame systems under test.

4.4 RQ4: Contributions of Feature Knowledge and Data Flow Graphs

RQ4 examines the individual contributions of feature knowledge and data flow graphs to SQL-to-DataFrame translation.

Experimental Setup. Following the RQ1 configuration, we conduct an ablation study to assess the impact of each component. We compare three configurations: (1) **LLM-only**: TDiFf without feature knowledge and data flow graphs; (2) **TDiFf⁻**: TDiFf without data flow graphs; and (3) the full **TDiFf** framework. After each run, we evaluate the generated DataFrame test cases using two metrics. The **Syntactic Correctness Ratio** is the proportion of test cases that execute without errors, whereas the **Semantic Correctness Ratio** is the proportion whose outputs match those of

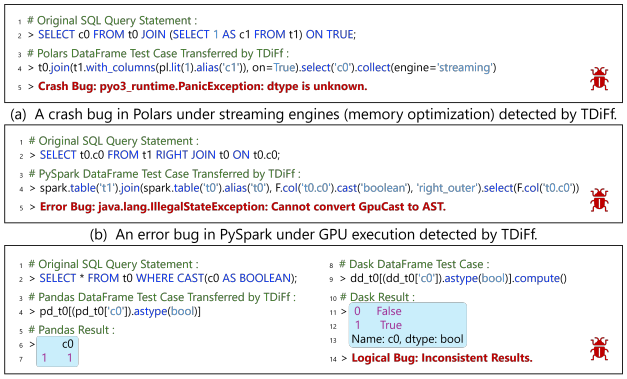


Figure 9: Three examples of true positive bugs.

the original SQL queries. To reduce potential bias toward a specific LLM, we repeat the ablation study using the DeepSeek API (version `DeepSeek-V3.2-Thinking 2025/12/01`) [43], with the same temperature setting (0.0) used for the OpenAI GPT-5-mini API [52].

Experimental Results. Table 10 reports the syntactic and semantic correctness of translated DataFrame test cases across the three categories. The full TDiFf achieves the highest correctness in all categories. Average syntactic correctness increases from 0.70 with LLM-only to 1.00 with TDiFf$^-$ and remains at 1.00 with TDiFf, indicating that feature knowledge eliminates the observed syntactic errors. Average semantic correctness increases from 0.53 to 0.72 with TDiFf$^-$ and further to 0.80 with TDiFf, highlighting the additional contribution of data flow graphs to semantic alignment.

Similar improvement trends are observed with both GPT-5-mini and DeepSeek-V3.2-Thinking, indicating that the effectiveness of TDiFf is not tied to a specific LLM. These results show that feature knowledge improves syntactic correctness, while data flow graph guidance preserves semantic fidelity. Together, they enable TDiFf to generate reliable, executable test cases for differential testing.

Answer to RQ4: Feature knowledge and data flow graphs jointly improve both the syntactic and semantic correctness of transferred test cases.

5 Discussion

5.1 Case Study

Case Study of True Positives. We present three representative bugs uncovered by TDiFf and missed by all baseline approaches. Figure 9(a) shows a panic crash in *Polars* (*issue #25544* [77]) triggered by the interaction between `DataFrame.join()` and `DataFrame.with_columns()` under streaming execution, a memory optimization. This bug has been confirmed and fixed by developers.

Figure 9(b) shows an error in *PySpark* (*issue #13907* [78]) during GPU execution. The bug was caused by the GPU nested loop join handler when join conditions contained expressions not convertible to an AST (e.g., casting a column to boolean), and has been confirmed and fixed by developers.

Figure 9(c) illustrates a logical bug in *Dask* (*issue #12172* [73]), where indexing a DataFrame with a boolean-cast value produces inconsistent results between *Pandas* and *Dask* distributed execution. This behavioral discrepancy has been confirmed by developers.

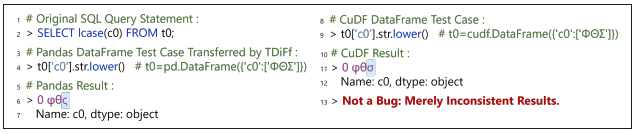


Figure 10: An example of a false positive bug.

Overall, these cases demonstrate that TDiFf can uncover bugs that arise from multi-API interactions or optimized execution paths. They cover all three DataFrame categories, span three major bug types, and involve diverse optimizations, including memory optimization, GPU execution, and distributed execution.

Case Study of False Positives. Although TDiFf applies a set of normalization and tolerance rules to reduce false positives, as detailed in Section 3.5, it may still produce a small number of false positives. In our six-hour run, TDiFf reported two false positives.

Figure 10 shows one such case in *CuDF* (*issue #21137* [72]), where a subtle edge condition involving the lowercase sigma character at the end of a word led to a reported discrepancy that the developer ultimately classified as `Won't Fix`.

Another case involves *Polars* (*issue #25098* [75]), which was judged to be similar to a previously reported but unresolved issue and therefore was not counted as a newly confirmed bug. These instances are rare and do not affect the overall utility of TDiFf.

5.2 Validation of Semantic Equivalence

To further validate whether consistent results indeed imply true semantic equivalence, we manually inspected 100 randomly sampled translation pairs whose execution results were consistent for each DataFrame category. The inspection confirmed that semantic equivalence holds in all sampled cases.

5.3 Threats to Validity

Semantic Correctness Validity. We assess semantic correctness by comparing the outputs of translated DataFrame test cases with those of the original SQL queries. However, identical outputs do not guarantee semantic equivalence, while differences between DBMSs and DataFrame systems may cause equivalent computations to produce different outputs. We mitigate these threats to the extent possible through manual sampling and validation, as described in Section 5.2. Our analysis suggests that such cases are rare and unlikely to materially affect our conclusions.

DataFrame Test Scope Validity. TDiFf translates Data Query Language (DQL) statements produced by SQL query generators into query-related DataFrame API sequences for bug detection. Therefore, it may not cover DataFrame APIs that are not naturally expressible as SQL queries. This threat concerns the scope of API behaviors exercised by TDiFf, while our evaluation remains valid for the query-related DataFrame test cases targeted by this work.

Bug Ground Truth Validity. Our evaluation measures bug-finding yield rather than recall because no complete ground truth exists for latent bugs. We therefore rely on developer confirmations, bug fixes, and baseline comparisons as evidence of practical effectiveness.

SQL Dialect Validity. Our evaluation uses SQL queries in the DuckDB dialect, while other DBMS dialects, such as PostgreSQL, may exercise different SQL features and reveal additional bugs. This may limit the generalizability of our findings to other SQL dialects, which warrants further evaluation in future work.

Table 10: Syntactic and semantic correctness ratios across three DataFrame categories. White rows report results obtained using GPT-5-mini as the base LLM, whereas gray rows report results obtained using DeepSeek-V3.2-Thinking. *Parenthesized values* in the TDiFf⁻ and TDiFf columns denote the percentage improvements over LLM-only and TDiFf⁻, respectively.

DataFrame Category	Syntactic Correctness Ratio			Semantic Correctness Ratio		
	LLM-only	TDiFf⁻	TDiFf	LLM-only	TDiFf⁻	TDiFf
Pandas-compatible	0.88	1.00 (+14%)	1.00	0.68	0.75 (+10%)	0.82 (+9%)
	0.66	1.00 (+52%)	1.00	0.46	0.68 (+48%)	0.80 (+18%)
PySpark	0.72	0.99 (+38%)	0.99	0.58	0.81 (+40%)	0.89 (+10%)
	0.69	0.99 (+43%)	0.99	0.56	0.79 (+41%)	0.89 (+13%)
Polars	0.51	0.99 (+94%)	1.00	0.32	0.60 (+88%)	0.67 (+12%)
	0.37	0.97 (+162%)	0.98	0.24	0.58 (+142%)	0.68 (+17%)
Average	0.70	1.00 (+43%)	1.00	0.53	0.72 (+36%)	0.80 (+11%)
	0.58	0.99 (+71%)	0.99	0.42	0.68 (+62%)	0.79 (+16%)

6 Related Work

DataFrame Systems and Benchmarking. DataFrame systems have become essential infrastructure for modern data analytics by providing expressive abstractions for structured data manipulation [68]. While *Pandas* established the standard for single-machine Python data analysis [48], *Dask*, *CuDF*, *Modin*, *PySpark*, and *Polars* improve scalability through parallelism, lazy evaluation, and hardware acceleration [2, 21, 25, 49, 58, 64]. Moreover, the complex optimizations that improve performance can introduce subtle behavioral inconsistencies, motivating systematic testing. Several benchmarks evaluate DataFrame performance: *FuzzyData* [61] generates workflows for stress testing and comparative analysis, while Bento [49] evaluates popular systems on single-machine data-preparation workloads. These efforts focus on performance rather than bug detection.

DBMS Testing. Generation-based testing has proved effective in detecting DBMS bugs. SQLsmith [65] generates random `SELECT` statements to uncover crashes. NoREC [62] compares optimizable queries with non-optimizable variants, while TLP [63] partitions query results to detect optimization bugs. CODDTest [86] targets constant folding through result-equivalent expression pairs. These tools produce SQL queries with rich syntax and semantics. TDiFf reuses this maturity by translating them into DataFrame test cases instead of rebuilding generation logic.

Python Testing. Automated Python test generation includes evolutionary and heuristic techniques [1, 34]. *Pynguin* [45] provides an extensible unit-test framework, and empirical studies show that search-based algorithms such as DynaMOSA [56] outperform random testing [46]. However, general-purpose techniques often produce invalid or trivial tests for DataFrame APIs, whereas TDiFf derives semantically rich multi-API tests from SQL queries. This enables it to exercise interactions and optimization paths that individual unit tests often miss.

API Testing. Existing API testing spans REST APIs and library APIs. RESTler [3] generates stateful request sequences from REST specifications, GraphFuzz [31] models library API sequences as dataflow graphs, and DeepREL [11] identifies related deep-learning APIs using syntactic and semantic information. These approaches often rely on incomplete specifications or inferred relations and therefore lack high-level semantic guidance. In contrast, TDiFf uses SQL queries as semantic specifications to generate equivalent DataFrame test cases.

LLM for Testing. LLMs increasingly automate testing tasks, particularly unit-test generation and fuzzing [33, 39, 79, 85]. ChatUniTest [8] combines adaptive prompting with generation, validation, and repair to improve coverage. LLM-generated Python tests can achieve coverage comparable to *Pynguin* but still face correctness and completeness challenges [5, 38]; LLM-assisted type tracing further improves *Pynguin* [41]. For fuzzing, Fuzz4All [83] generates and mutates inputs across programming and formal languages, while recent studies integrate LLMs into DBMS fuzzing [37]. Unlike direct generation, TDiFf translates semantically rich SQL into DataFrame API sequences to expose bugs caused by multi-API interactions and optimized execution.

SQL Dialect Translation. Sedar [27], QTRAN [42], and CrackSQL [88] translate between SQL dialects under the same declarative model. TDiFf instead translates declarative SQL into procedural, object-oriented Python DataFrame API sequences rather than another SQL dialect, requiring richer semantic guidance. It uses query-plan-derived data flow graphs to preserve semantic consistency across interfaces. This cross-interface problem cannot be reduced to syntactic dialect mappings. Nevertheless, existing dialect translators can complement TDiFf by first converting queries from other DBMSs, such as PostgreSQL, into DuckDB SQL for subsequent SQL-to-DataFrame translation.

7 Conclusion

We presented TDiFf, a DataFrame testing framework using feature knowledge and data flow graphs to guide LLM-based SQL-to-DataFrame translation. Over 99% of the generated test cases are valid, and TDiFf uncovers 35 unique, developer-confirmed, previously unknown bugs, outperforming state-of-the-art baselines by 2.5× to 14× in bug detection. These results demonstrate its effectiveness and practicality for automated DataFrame testing.

Acknowledgments

We thank the anonymous reviewers for their valuable and insightful feedback. This research was supported by the Natural Science Foundation of China (Grant No. 62272400) and the Fujian Provincial Natural Science Foundation of China (Grant No. 2025J010002). Rongxin Wu is the corresponding author and works as a member of Fujian Engineering Research Center of High-Performance Intelligent Computing Systems in Xiamen University.

Data Availability

The source code and bug list of TDiFf are available at https://doi.org/10.5281/zenodo.21393787.

References

[1] Majdah Alshammari, Mohammad A Mezher, and Khaled Al-utaibi. 2022. Automatic test data generation using genetic algorithm for python programs. In *2022 2nd International Conference On Computing And Information Technology (ICCIT)*. IEEE, 197–205. doi:10.1109/ICCIT52419.2022.9711607

[2] Michael Armbrust, Tathagata Das, Aaron Davidson, Ali Ghodsi, Andrew Or, Josh Rosen, Ion Stoica, Patrick Wendell, Reynold Xin, and Matei Zaharia. 2015. Scaling spark in the real world: performance and usability. *Proceedings of the VLDB Endowment* 8, 12 (2015), 1840–1843. doi:10.14778/2824032.2824080

[3] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. Restler: Stateful rest api fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 748–758. doi:10.1109/ICSE.2019.00083

[4] Alan Beaulieu. 2009. Learning SQL: master SQL fundamentals. https://dl.acm.org/doi/abs/10.5555/1611255

[5] Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. 2024. Unit test generation using generative ai: A comparative performance analysis of autogeneration tools, 2024. *URL https://arxiv. org/abs/2312.10622* 2, 2.3 (2024), 5.

[6] Alex Broihier, Stefanos Baziotis, Daniel Kang, and Charith Mendis. 2025. Pandas-Bench: A Benchmark for the Pandas API. *arXiv preprint arXiv:2506.02345* (2025). doi:10.48550/arXiv.2506.02345

[7] Surajit Chaudhuri and Umeshwar Dayal. 1997. An overview of data warehousing and OLAP technology. *ACM Sigmod record* 26, 1 (1997), 65–74. doi:10.1145/248603.248616

[8] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. Chatunitest: A framework for llm-based test generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 572–576. doi:10.1145/3663529.3663801

[9] Yuting Chen, Ting Su, and Zhendong Su. 2019. Deep differential testing of JVM implementations. *ICSE'19* (2019), 1257–1268. doi:10.1109/ICSE.2019.00127

[10] George Colliat. 1996. OLAP, relational, and multidimensional database systems. *ACM Sigmod Record* 25, 3 (1996), 64–69. doi:10.1145/234889.234901

[11] Yinlin Deng, Chenyuan Yang, Anjiang Wei, and Lingming Zhang. 2022. Fuzzing deep-learning libraries via automated relational api inference. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 44–56. doi:10.1145/3540250.3549085

[12] Zizhuang Deng, Guozhu Meng, Kai Chen, Tong Liu, Lu Xiang, and Chunyang Chen. 2023. Differential testing of cross deep learning framework {APIs}: Revealing inconsistencies and vulnerabilities. *USENIX Security'23* (2023), 7393–7410.

[13] The Apache Spark development team. 2025. Apache Spark: A unified analytics engine for large-scale data processing. https://github.com/apache/spark.

[14] The Apache Spark development team. 2025. PySpark Documentation. https://spark.apache.org/docs/latest/api/python/index.html.

[15] The Dask development team. 2025. Dask DataFrame Documentation. https://docs.dask.org/en/stable/dataframe.html.

[16] The DuckDB development team. 2025. DuckDB SQLsmith extension repository. https://github.com/duckdb/duckdb-sqlsmith.

[17] The Dask development team. 2025. Parallel computing with task scheduling. https://github.com/dask/dask.

[18] The Modin development team. 2025. Modin Documentation. https://modin.readthedocs.io/en/stable/.

[19] The Modin development team. 2025. Modin: Scale your Pandas workflows by changing a single line of code. https://github.com/modin-project/modin.

[20] The NVIDIA RAPIDS development team. 2025. cuDF - GPU DataFrame Library. https://github.com/rapidsai/cudf.

[21] The NVIDIA RAPIDS development team. 2025. CuDF Documentation. https://docs.rapids.ai/api/cudf/stable/.

[22] The NVIDIA RAPIDS development team. 2025. Spark RAPIDS plugin: accelerate Apache Spark with GPUs. https://github.com/NVIDIA/spark-rapids.

[23] The Polars development team. 2025. Polars: Extremely fast Query Engine for DataFrames, written in Rust. https://github.com/pola-rs/polars.

[24] The Polars development team. 2025. Polars SQL Compatibility Issues. https://docs.pola.rs/user-guide/sql/intro/#compatibility.

[25] The Polars development team. 2025. Polars User Guide. https://docs.pola.rs/.

[26] Sebastian Elbaum, Hui Nee Chin, Matthew B Dwyer, and Jonathan Dokulil. 2006. Carving differential unit test cases from system test cases. In *Proceedings of the 14th ACM SIGSOFT international symposium on Foundations of software engineering*. 253–264. doi:10.1145/1181775.1181806

[27] Jingzhou Fu, Jie Liang, Zhiyong Wu, and Yu Jiang. 2024. Sedar: Obtaining high-quality seeds for dbms fuzzing via cross-dbms sql transfer. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639210

[28] Ying Fu, Zhiyong Wu, Yuanliang Zhang, Jie Liang, Jingzhou Fu, Yu Jiang, Shanshan Li, and Xiangke Liao. 2025. THANOS: DBMS Bug Detection via Storage Engine Rotation Based Differential Testing. *ICSE'25* (2025), 1–12. doi:10.1109/ICSE55347.2025.00001

[29] Georgi Gerganov. 2025. Graphviz to ASCII converter using Graph::Easy. https://github.com/ggerganov/dot-to-ascii.

[30] Goetz Graefe. 2002. Volcano/spl minus/an extensible and parallel query evaluation system. *IEEE Transactions on Knowledge and Data Engineering* 6, 1 (2002), 120–135. doi:10.1109/69.273032

[31] Harrison Green and Thanassis Avgerinos. 2022. Graphfuzz: Library api fuzzing with lifetime-aware dataflow graphs. In *Proceedings of the 44th International Conference on Software Engineering*. 1070–1081. doi:10.1145/3510003.3510228

[32] Hao Guan, Ying Xiao, Jiaying Li, Yepang Liu, and Guangdong Bai. 2023. A comprehensive study of real-world bugs in machine learning model optimization. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 147–158. doi:10.1109/ICSE48619.2023.00024

[33] Ishrak Hayet, Adam Scott, and Marcelo d'Amorim. 2024. Chatassert: Llm-based test oracle generation with external tools assistance. *IEEE Transactions on Software Engineering* 51, 1 (2024), 305–319. doi:10.1109/TSE.2024.3519159

[34] Josie Holmes, Iftekhar Ahmed, Caius Brindescu, Rahul Gopinath, He Zhang, and Alex Groce. 2020. Using relative lines of code to guide automated test generation for python. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 29, 4 (2020), 1–38. doi:10.1145/3408896

[35] Jiaxin Hu, Shaowei Chen, and Rongxin Wu. 2026. TDiFf: Detecting Bugs in DataFrame Systems via Transferred DBMS Test Cases. doi:10.5281/zenodo.21393787

[36] Ahmad Humayun. 2026. DAGger: DAG-Based Workload Generation for Testing Data-Intensive Computing Frameworks. https://github.com/ahmayun/dag-fuzzer-generalized.

[37] Yu Jiang, Jie Liang, Fuchen Ma, Yuanliang Chen, Chijin Zhou, Yuheng Shen, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Shanshan Li, et al. 2024. When fuzzing meets llms: Challenges and opportunities. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 492–496. doi:10.1145/3663529.3663784

[38] Medlen Jiri, Bari Emese, and Patrick Medlen. 2024. Leveraging Large Language Models for Python Unit Test. In *2024 IEEE International Conference on Artificial Intelligence Testing (AITest)*. IEEE, 95–100. doi:10.1109/AITest62860.2024.00020

[39] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323. doi:10.1109/ICSE48619.2023.00194

[40] Kavi, Buckles, and Bhat. 1986. A formal definition of data flow graph models. *IEEE Transactions on computers* 100, 11 (1986), 940–948. doi:10.1109/TC.1986.1676696

[41] Lukas Krodinger, Stephan Lukasczyk, and Gordon Fraser. 2025. Combining Type Inference and Automated Unit Test Generation for Python. *arXiv preprint arXiv:2507.01477* (2025). doi:10.48550/arXiv.2507.01477

[42] Li Lin, Qinglin Zhu, Hongqiao Chen, Zhuangda Wang, Rongxin Wu, and Xiaoheng Xie. 2025. QTRAN: Extending Metamorphic-Oracle Based Logical Bug Detection Techniques for Multiple-DBMS Dialect Support. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 731–752. doi:10.1145/3728908

[43] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. 2025. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556* (2025). doi:10.48550/arXiv.2512.02556

[44] Chunwei Liu, Anna Pavlenko, Matteo Interlandi, and Brandon Haynes. 2023. A deep dive into common open formats for analytical dbmss. *Proceedings of the VLDB Endowment* 16, 11 (2023), 3044–3056. doi:10.14778/3611479.3611507

[45] Stephan Lukasczyk and Gordon Fraser. 2022. Pynguin: Automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*. 168–172. doi:10.1145/3510454.3516829

[46] Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2023. An empirical study of automated unit test generation for Python. *Empirical Software Engineering* 28, 2 (2023), 36. doi:10.1007/s10664-022-10248-w

[47] William M McKeeman. 1998. Differential testing for software. *Digital Technical Journal* 10, 1 (1998), 100–107. https://api.semanticscholar.org/CorpusID:14018070

[48] Wes McKinney et al. 2010. Data structures for statistical computing in Python. *scipy* 445, 1 (2010), 51–56. doi:10.25080/Majora-92bf1922-00a

[49] Angelo Mozzillo, Luca Zecchini, Luca Gagliardelli, Adeel Aslam, Sonia Bergamaschi, Giovanni Simonini, et al. 2025. Evaluation of Dataframe Libraries for Data Preparation on a Single Machine. In *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*. doi:10.48786/edbt.2025.27

[50] Siu Lung Ng and Fethi Rabhi. 2022. A Data Analytics Architecture for the Exploratory Analysis of High-Frequency Market Data. In *International Workshop on Enterprise Applications, Markets and Services in the Finance Industry*. Springer, 1–16. doi:10.1007/978-3-031-31671-5_1

[51] Open2C, Nezar Abdennur, Geoffrey Fudenberg, Ilya M Flyamer, Aleksandra A Galitsyna, Anton Goloborodko, Maxim Imakaev, and Sergey Venev. 2024. Bioframe: operations on genomic intervals in Pandas dataframes. *Bioinformatics* 40, 2 (2024),

btae088. doi:10.1093/bioinformatics/btae088
[52] OpenAI. 2025. GPT-5 mini: A faster, cost-efficient version of GPT-5 for well-defined tasks. https://platform.openai.com/docs/models/gpt-5-mini.
[53] The pandas development team. 2025. pandas: A Powerful Python Data Analysis Toolkit. https://github.com/pandas-dev/pandas.
[54] The pandas development team. 2025. Pandas Documentation. https://pandas.pydata.org/.
[55] The pandasql development team. 2016. pandasql: sqldf for pandas. https://pypi.org/project/pandasql/.
[56] Annibale Panichella, Fitsum Meshesha Kifetew, and Paolo Tonella. 2017. Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. *IEEE Transactions on Software Engineering* 44, 2 (2017), 122–158. doi:10.1109/TSE.2017.2663435
[57] Devin Petersohn. 2021. *Dataframe systems: Theory, architecture, and implementation*. Ph. D. Dissertation. University of California, Berkeley.
[58] Devin Petersohn, Stephen Macke, Doris Xin, William Ma, Doris Lee, Xiangxi Mo, Joseph E Gonzalez, Joseph M Hellerstein, Anthony D Joseph, and Aditya Parameswaran. 2020. Towards scalable dataframe systems. *arXiv preprint arXiv:2001.00888* (2020). doi:10.48550/arXiv.2001.00888
[59] Devin Petersohn, Dixin Tang, Rehan Durrani, Areg Melik-Adamyan, Joseph E Gonzalez, Anthony D Joseph, and Aditya G Parameswaran. 2021. Flexible rule-based decomposition and metadata independence in modin: a parallel dataframe system. *Proceedings of the VLDB Endowment* 15, 3 (2021). doi:10.14778/3494124.3494152
[60] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB. In *Proceedings of the 2019 International Conference on Management of Data*. doi:10.1145/3299869.3320212
[61] Mohammed Suhail Rehman and Aaron Elmore. 2022. Fuzzydata: a scalable workload generator for testing dataframe workflow systems. In *Proceedings of the 2022 workshop on 9th International Workshop of Testing Database Systems*. 17–24. doi:10.1145/3531348.3532178
[62] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1140–1152. doi:10.1145/3368089.3409710
[63] Manuel Rigger and Zhendong Su. 2020. Finding bugs in database systems via query partitioning. *Proceedings of the ACM on Programming Languages* OOPSLA (2020). doi:10.1145/3428279
[64] Matthew Rocklin et al. 2015. Dask: Parallel computation with blocked algorithms and task scheduling.. In *SciPy*. 126–132. doi:10.25080/Majora-7b98e3ed-013
[65] Andreas Seltenreich. 2025. SQLsmith: A random SQL query generator. https://github.com/anse1/sqlsmith.
[66] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. 2025. A survey on employing large language models for text-to-sql tasks. *Comput. Surveys* 58, 2 (2025), 1–37. doi:10.1145/3737873
[67] Abraham Silberschatz, Henry F Korth, Shashank Sudarshan, et al. 2002. *Database system concepts*. Vol. 5. Mcgraw-hill New York. https://dl.acm.org/doi/10.5555/551244
[68] Phanwadee Sinthong and Michael J Carey. 2021. Exploratory data analysis with database-backed dataframes: A case study on airbnb data. In *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 3119–3129. doi:10.1109/BigData52589.2021.9671603
[69] The sqlglot development team. 2025. SQLGlot: Python SQL Parser and Transpiler. https://github.com/tobymao/sqlglot.
[70] Polars Team. 2026. Polars Unit Test for DataFrame.filter API. https://github.com/pola-rs/polars/blob/main/py-polars/tests/unit/operations/test_filter.py.
[71] Polars Team. 2026. Polars Unit Test for DataFrame.join API. https://github.com/pola-rs/polars/blob/main/py-polars/tests/unit/operations/test_join.py.
[72] TDiFf Team. 2026. CuDF Issue 21137 Detected by TDiFf. https://github.com/rapidsai/cudf/issues/21137.
[73] TDiFf Team. 2026. Dask Issue 12172 Detected by TDiFf. https://github.com/dask/dask/issues/12172.
[74] TDiFf Team. 2026. Polars Issue 25072 Detected by TDiFf. https://github.com/pola-rs/polars/issues/25072.
[75] TDiFf Team. 2026. Polars Issue 25098 Detected by TDiFf. https://github.com/pola-rs/polars/issues/25098.
[76] TDiFf Team. 2026. Polars Issue 25211 Detected by TDiFf. https://github.com/pola-rs/polars/issues/25211.
[77] TDiFf Team. 2026. Polars Issue 25544 Detected by TDiFf. https://github.com/pola-rs/polars/issues/25544.
[78] TDiFf Team. 2026. PySpark Issue 13907 Detected by TDiFf. https://github.com/NVIDIA/spark-rapids/issues/13907.
[79] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. doi:10.1109/TSE.2024.3368208
[80] David Waterworth. 2026. Polars Issue 19772. https://github.com/pola-rs/polars/issues/19772.
[81] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837. doi:10.52202/068431-1800
[82] Yifan Wu. 2020. Is a dataframe just a table?. In *10th Workshop on Evaluation and Usability of Programming Languages and Tools (PLATEAU 2019)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 6–1. doi:10.4230/OASIcs.PLATEAU.2019.6
[83] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13. doi:10.1145/3597503.3639121
[84] Tao Xie, Kunal Taneja, Shreyas Kale, and Darko Marinov. 2007. Towards a framework for differential unit testing of object-oriented programs. In *Second International Workshop on Automation of Software Test (AST'07)*. IEEE, 5–5. doi:10.1109/AST.2007.15
[85] Shengcheng Yu, Chunrong Fang, Yuchen Ling, Chentian Wu, and Zhenyu Chen. 2023. Llm for test script generation and migration: Challenges, capabilities, and opportunities. In *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*. IEEE, 206–217. doi:10.1109/QRS60937.2023.00029
[86] Chi Zhang and Manuel Rigger. 2025. Constant Optimization Driven Database System Testing. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–24. doi:10.1145/3709674
[87] Chi Zhang, Linzhang Wang, and Manuel Rigger. 2024. Finding cross-rule optimization bugs in datalog engines. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 110–136. doi:10.1145/3649815
[88] Wei Zhou, Yuyang Gao, Xuanhe Zhou, and Guoliang Li. 2025. Cracking SQL Barriers: An LLM-based Dialect Translation System. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26. doi:10.1145/3725278
[89] Zhide Zhou, Zhilei Ren, Guojun Gao, and He Jiang. 2021. An empirical study of optimization bugs in GCC and LLVM. *Journal of Systems and Software* 174 (2021), 110884. doi:10.1016/j.jss.2020.110884

Received 2026-03-26; accepted 2026-06-18